

ÉCOLE MAROCAINE DES SCIENCES DE L'INGÉNIEUR (EMSI)

FILIÈRE : DDSIG1

RAPPORT DE PROJET ACADÉMIQUE DÉTAILLÉ

Plateforme de Gestion des Absences Multi-Rôles
Spécifications Fonctionnelles V2, Architecture JPA, Sessions & Cookies

Membres d'équipe :

Nizar Belkhadir
Adam Bibilou
Adam Asaas
Chafiq Mwiye

Présenté devant :

Pr. Houssam BAZZA

Module :

JEE et Génie Logiciel

Année Universitaire : 2025/2026

Table des matières

1	Introduction et Contexte du Projet	3
1.1	Problématique de l'assiduité universitaire	3
1.2	Choix Technologique : Java EE 8 pur et JPA	3
2	Cahier des Charges et Spécifications Fonctionnelles	3
2.1	Description des Rôles Applicatifs	3
2.1.1	Espace Administration (Admin)	3
2.1.2	Espace Enseignant (Professeur)	4
2.1.3	Espace Étudiant	4
2.2	Cycle de Vie des Demandes de Régularisation	4
3	Partie Conception (Modélisation UML)	4
3.1	Diagramme de Cas d'Utilisation (Use Case Diagram)	4
3.2	Diagramme de Classes (Class Diagram)	4
3.3	Diagramme de Séquence (Sequence Diagram)	6
4	Patrons de Conception (Design Patterns) Appliqués	6
4.1	Modèle-Vue-Contrôleur (MVC)	6
4.2	Front Controller (Contrôleur Frontal)	6
4.3	Data Access Object (DAO)	6
4.4	Singleton (Unique Instance)	8
4.5	ThreadLocal Context (Isolation par Thread)	8
5	Infrastructure, Persistance et Couche Modèle	9
5.1	Conteneurisation Database (PostgreSQL et Docker)	9
5.2	Configuration de JPA (<code>persistence.xml</code>)	9
5.3	Implémentation des Classes du Domaine (Modèles JPA)	10
6	Gestion du Cycle de Vie JPA et EntityManagers	11
6.1	Le Listener de Démarrage (<code>JPAInitializer.java</code>)	11
6.2	Thread Isolation des Managers (<code>JPAUtil.java</code>)	11
7	Couche de Persistance : Implémentation des DAOs	12
7.1	Gestion Manuelle des Transactions	12
7.2	Le Patron Data Access Object (DAO)	12
8	Couche Contrôleur : La Servlet Front-Controller	12
8.1	Vérification des Rôles et Session Gate	13
8.2	Structure Majeure du Servlet de Contrôle	13
9	Gestion d'État : Sessions & Cookies	14
9.1	Statelessness HTTP et HttpSession	14
9.2	Persistance et Remplissage via Cookies	14
10	Téléversement de Fichiers & Exportations	15
10.1	Téléversement avec Multipart Config	15
10.2	Exportation CSV avec Signature BOM	15

11 Interface Utilisateur (UI) & CSS Glassmorphism	15
11.1 Philosophie du Design System	15
11.2 Filtrage client temps réel (Vanilla JS)	16
12 Structure Détaillée des Vues (JSPs)	16
12.1 Espace Étudiant - student-dashboard.jsp	16
12.2 Espace Enseignant - prof-dashboard.jsp	17
12.3 Espace Administration - admin-requests.jsp	17
13 Déploiement et Configuration	18
13.1 Configuration de déploiement (web.xml)	18
13.2 Maven pom.xml	18
14 Scénarios d'Exécution et Recette de Tests	18
14.1 Test 1 : Authentification multi-rôle	18
14.2 Test 2 : Mémorisation de session par Cookies	18
14.3 Test 3 : Saisie d'appel interactif par le Professeur	18
14.4 Test 4 : Tableau de bord Étudiant et suivi personnel	18
14.5 Test 5 : Modération administrative des justificatifs et réclamations	21
15 Conclusion et Perspectives	21

1 Introduction et Contexte du Projet

1.1 Problématique de l'assiduité universitaire

Dans le cadre de la gestion académique au sein des établissements d'enseignement supérieur, le suivi de l'assiduité des étudiants représente un facteur déterminant pour garantir la rigueur pédagogique et lutter contre le décrochage scolaire. La saisie manuelle des absences, l'archivage physique des justificatifs médicaux et le calcul fastidieux des statistiques d'assiduité sont des tâches chronophages pour le corps administratif.

Un système défaillant ou obsolète engendre de nombreux problèmes :

- **Perte d'informations** : Perte de justificatifs papier déposés auprès de l'administration.
- **Erreurs de saisie** : Incohérences lors du report manuel des absences saisies par les enseignants.
- **Manque de réactivité** : Les étudiants ne sont informés de leurs absences qu'en fin de semestre, rendant impossible toute contestation rapide ou rattrapage.

Ce projet vise à relever ces défis en concevant une plateforme interactive multi-rôles permettant d'interconnecter directement l'administration scolaire, les professeurs et les étudiants, assurant un traitement transparent et collaboratif de l'assiduité.

1.2 Choix Technologique : Java EE 8 pur et JPA

Afin d'acquérir une compréhension profonde des mécanismes internes des architectures logicielles d'entreprise, nous avons opté pour le développement d'une application en **Java EE 8 pur (Jakarta EE 8)** sans recours aux frameworks d'auto-configuration de type Spring Boot. Cette approche permet de manipuler directement :

- Les cycles de vie des **Servlets** HTTP.
- Le rendu dynamique par **JSP** (JavaServer Pages) avec Expression Language (EL) et JSTL.
- La gestion de persistance par **JPA** (fournisseur Hibernate) avec requêtes JPQL.
- La conteneurisation de la base de données relationnelle via **PostgreSQL 15** sous Docker.

2 Méthodologie de Gestion de Projet (Scrum)

Dans le cadre de la planification et de l'exécution de notre plateforme, nous avons adopté la méthodologie agile **Scrum**. Ce cadre de travail nous a permis de structurer nos développements en cycles courts, de réagir rapidement aux contraintes techniques et de garantir un incrément logiciel de haute qualité.

2.1 Composition de l'Équipe Scrum

L'équipe est composée des rôles agiles suivants :

- **Product Owner (PO)** : Représente les besoins de l'administration scolaire et du corps enseignant (EMSI). Il est responsable de la spécification et de la priorisation des exigences (User Stories).

- **Scrum Master (SM) : Nizar Belkhadir** — Assure la bonne application des pratiques Scrum, anime les rituels agiles, résout les blocages techniques (Git, Docker, etc.) et maintient la vélocité de l'équipe.
- **Équipe de Développement (Development Team) : Nizar, Adam B., Adam A., Chafiq**. Équipe pluridisciplinaire responsable de la conception UML, du développement des composants front-end (JSP, CSS), back-end (Servlets, DAOs) et du déploiement.

2.2 Découpage et Objectifs des Sprints

Le projet a été développé sur une durée globale de trois semaines, découpée en trois Sprints successifs d'une semaine chacun :

1. Sprint 1 : Conception, Persistance et Charte Graphique

Ce premier sprint a permis d'analyser le besoin et de concevoir les diagrammes UML (Cas d'utilisation, Classes, Séquence). Nous avons initialisé le projet Maven, configuré la base de données PostgreSQL 15 sous Docker et mappé les entités JPA persistantes (**Etudiant**, **Absence**, etc.). Nous avons également mis en place l'architecture de persistance isolée (**JPAUtil** et **ThreadLocal**) et créé la charte graphique moderne en CSS.

2. Sprint 2 : Cœur Applicatif et Sécurité des Accès

L'objectif de ce sprint était d'implémenter le moteur de traitement. Nous avons écrit la Servlet unique de routage (**AbsenceServlet**) appliquant le patron Front Controller, géré les sessions utilisateurs (**HttpSession**), et conçu la feuille d'appel interactive avec enregistrement groupé pour les professeurs, ainsi que le registre global d'administration et l'export CSV.

3. Sprint 3 : Workflows de Modération, Tests et Polissage

Durant le dernier sprint, nous avons développé le dashboard étudiant et ses statistiques d'assiduité, les workflows d'upload de justificatifs médicaux, et les réclamations. Nous avons configuré la mémorisation de connexion par Cookies « Remember Me » et le filtrage client en Vanilla JS. Ce sprint s'est achevé par la mise en place de la suite de tests unitaires sous **JUnit 4** et la rédaction finale du rapport technique.

2.3 Rituels et Cérémonies Scrum

Pour maintenir la synchronisation de l'équipe, nous avons tenu rigoureusement les cérémonies agiles suivantes :

- **Sprint Planning** : Réunion hebdomadaire pour définir l'objectif du sprint et sélectionner les tâches du Backlog de produit.
- **Daily Stand-up** : Point quotidien de 15 minutes durant lequel chaque membre détaille ses actions de la veille, ses objectifs du jour et ses éventuels blocages.
- **Sprint Review** : Démo de fin de sprint devant le Product Owner pour valider la conformité de l'incrément développé.
- **Sprint Retrospective** : Réunion de fin de sprint pour identifier les points positifs et les axes d'amélioration.

TABLE 1 – Exemples clés de User Stories du Backlog de produit

Rôle	User Story	Objectif Métier
Professeur	En tant qu'enseignant, je veux faire l'appel sur une grille interactive	Enregistrer les absences rapidement.
Étudiant	En tant qu'étudiant, je veux téléverser un certificat médical PDF	Justifier mes absences auprès de l'administration.
Administrateur	En tant qu'administrateur, je veux exporter les absences en CSV	Analyser et éditer les données sous Microsoft Excel.
Étudiant	En tant qu'étudiant, je veux contester une absence induite	Permettre sa suppression par l'administrateur.

2.4 User Stories clés du Backlog

3 Cahier des Charges et Spécifications Fonctionnelles

3.1 Description des Rôles Applicatifs

L'architecture de l'application s'articule autour de trois rôles distincts, chacun disposant d'un espace applicatif sécurisé :

3.1.1 Espace Administration (Admin)

L'administrateur scolaire est le garant de la cohérence globale du système. Ses droits comprennent :

- Consultation du registre global de toutes les absences déclarées.
- Saisie manuelle d'une absence (par exemple, pour régulariser des séances particulières).
- Modération des demandes de justificatifs d'absence soumises par les étudiants.
- Modération des demandes de réclamation soumises par les étudiants.
- Exportation de l'historique complet des absences au format CSV pour Excel.

3.1.2 Espace Enseignant (Professeur)

L'enseignant est chargé de l'appel pour les cours qui lui sont attribués :

- Accès à la liste de ses cours et séances d'enseignement.
- Saisie rapide et interactive de la feuille d'appel pour une séance spécifique.
- Ajout automatique des absences pour les étudiants cochés absents, et suppression automatique si un étudiant est décoché.

3.1.3 Espace Étudiant

L'étudiant dispose d'un accès en lecture à ses propres données et peut formuler des demandes d'ajustement :

- Visualisation de ses statistiques personnelles (nombre total d'absences, absences justifiées, absences injustifiées).

- Dépôt d'une demande de justification (renseignant un motif textuel et une pièce justificative PDF/image optionnelle).
- Dépôt d'une réclamation pour signaler qu'il a été noté absent par erreur lors d'un cours où il était présent.

3.2 Cycle de Vie des Demandes de Régularisation

Pour assurer la modération sans supprimer les données arbitrairement, nous avons modélisé deux workflows distincts :

- **Flux de Justification** : L'étudiant soumet un motif et un fichier. L'absence passe au statut de justification `PENDING`. L'administrateur examine la demande : s'il l'accepte, le statut devient `ACCEPTED` et l'absence est considérée comme justifiée. S'il la refuse, le statut devient `REJECTED` et l'absence reste injustifiée.
- **Flux de Réclamation** : L'étudiant déclare sa présence induue. L'absence passe au statut de réclamation `PENDING`. L'administrateur valide : s'il accepte la réclamation, l'absence est supprimée de la base de données. S'il la rejette, le statut devient `REJECTED` et l'absence est maintenue.

4 Partie Conception (Modélisation UML)

4.1 Diagramme de Cas d'Utilisation (Use Case Diagram)

Le diagramme de cas d'utilisation synthétise les limites fonctionnelles du système et définit les rôles d'accès de chaque acteur (Étudiant, Professeur, Administrateur). Il permet d'illustrer visuellement les cas d'utilisation clés tels que la saisie d'absence, le dépôt de justificatif, et l'arbitrage des réclamations.

4.2 Diagramme de Classes (Class Diagram)

Le diagramme de classes représente le modèle conceptuel de données de notre domaine d'application. Chaque classe est directement mappée en entité JPA. Les relations modélisent l'association d'un étudiant à ses absences, d'une absence à une séance de cours spécifique, et d'un cours à un enseignant responsable.

4.3 Diagramme de Séquence (Sequence Diagram)

Le diagramme de séquence retrace les interactions temporelles entre les composants du système (Client JSP, Contrôleur Servlet, DAOs et Base de données PostgreSQL) lors d'un scénario type de soumission et de validation d'un justificatif d'absence.

5 Patrons de Conception (Design Patterns) Appliqués

Afin de concevoir une application modulaire, extensible et conforme aux bonnes pratiques du Génie Logiciel, plusieurs patrons de conception (*Design Patterns*) ont été implémentés au sein de l'architecture de la plateforme.

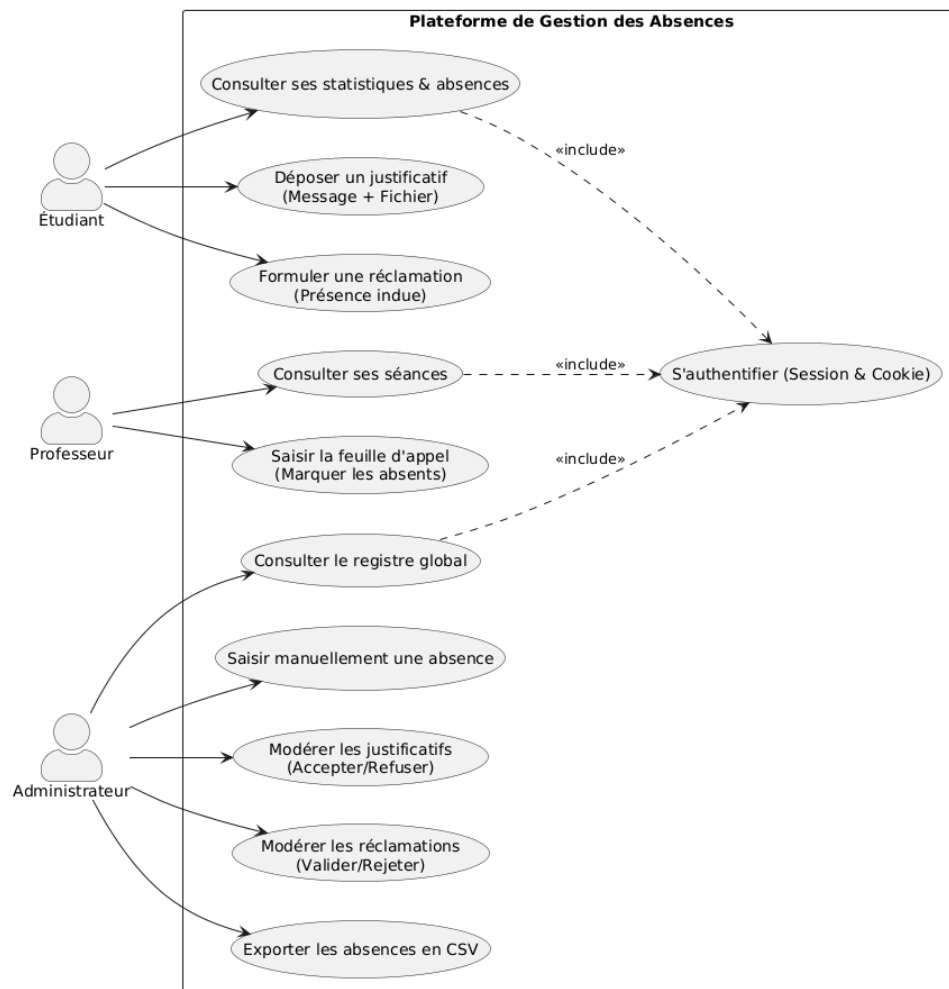


FIGURE 1 – Diagramme de Cas d'Utilisation - Plateforme de Gestion des Absences

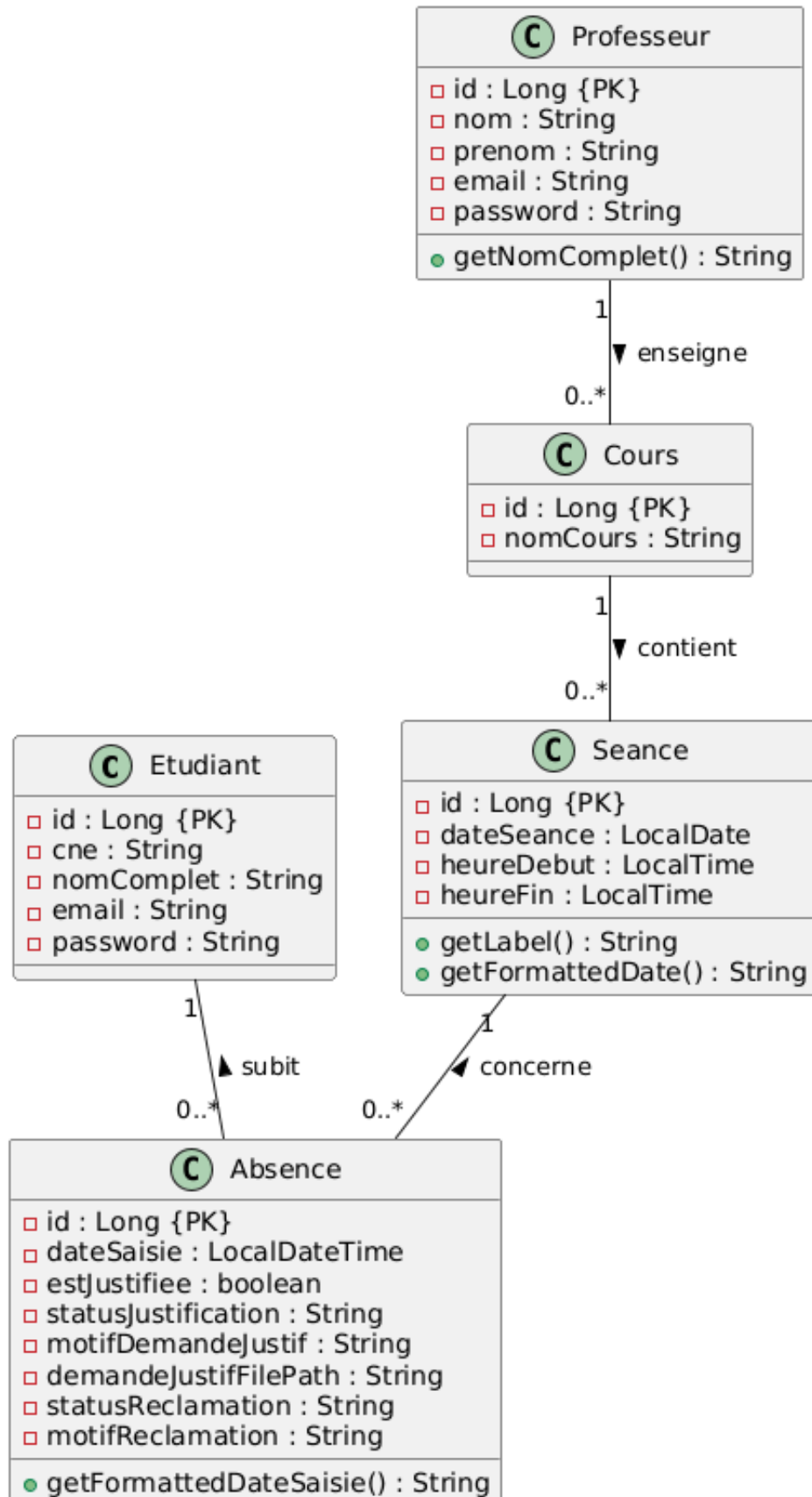


FIGURE 2 – Diagramme de Classes JPA - Modèle du Domaine

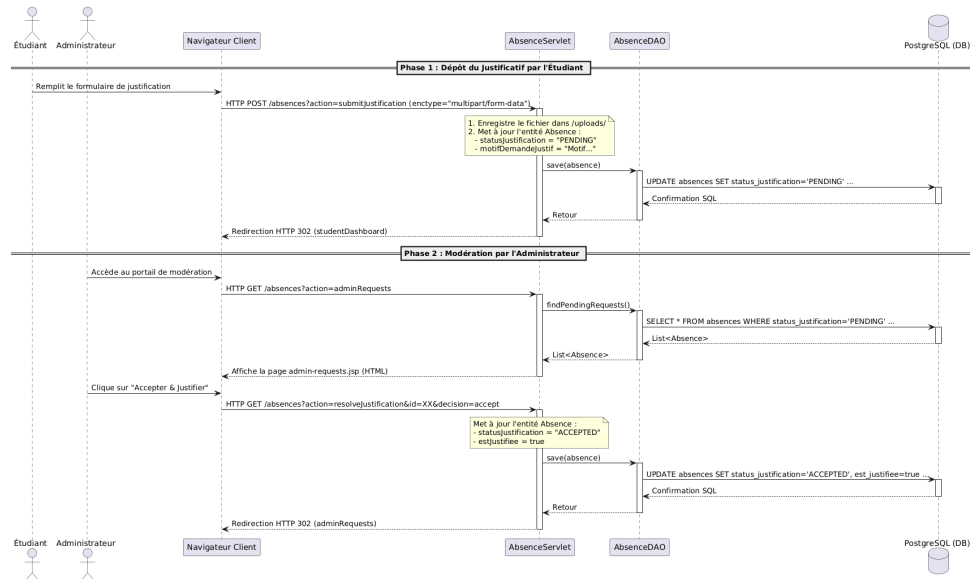


FIGURE 3 – Diagramme de Séquence - Traitement des Demandes et Persistance

5.1 Modèle-Vue-Contrôleur (MVC)

Le patron de conception architectural central est le **MVC**. Il sépare l'application en trois composants indépendants :

- **Le Modèle (Model)** : Représenté par les entités JPA (**Etudiant**, **Absence**, **Professeur**, **Seance**, **Cours**) et la couche de persistance. Il gère l'état et l'accès aux données de manière isolée.
- **La Vue (View)** : Représentée par les pages JSP sous **WEB-INF/views/**. Elles s'occupent uniquement du rendu graphique de l'interface et du dynamisme d'affichage via JSTL et EL, sans contenir de logique métier.
- **Le Contrôleur (Controller)** : Représenté par la servlet **AbsenceServlet** qui intercepte les requêtes HTTP, sollicite le modèle et sélectionne la vue appropriée.

5.2 Front Controller (Contrôleur Frontal)

Plutôt que d'associer un servlet distinct à chaque URL (ce qui alourdirait la maintenance et fragmenterait la gestion de la sécurité), la plateforme implémente le pattern **Front Controller**. Une servlet unique (**AbsenceServlet**) intercepte toutes les requêtes mappées sous **/absences**. Elle réalise les vérifications de session transversales (Session Gating) et de rôles, puis aiguille le traitement vers la méthode dédiée en fonction du paramètre **action** passé en paramètre d'URL.

5.3 Data Access Object (DAO)

Le pattern **DAO** isole complètement la logique d'accès à la base de données relationnelle du reste de l'application. La servlet contrôleur n'effectue jamais de requêtes JPQL en direct. Elle délègue ces opérations à des classes spécialisées (**AbsenceDAO**, **EtudiantDAO**, etc.). Cela permet de découpler le modèle physique de stockage de la logique applicative.

5.4 Singleton (Unique Instance)

La fabrique d'EntityManager (**EntityManagerFactory**) est une ressource lourde à initialiser. Nous appliquons le principe du **Singleton** encapsulé dans la classe **JPAUtil**. L'EntityManagerFactory est instancié une seule fois lors du démarrage du conteneur Tomcat par le listener **JPAInitializer** et partagé sous forme d'instance unique tout au long du cycle de vie de l'application.

5.5 ThreadLocal Context (Isolation par Thread)

Puisque l'EntityManager standard de JPA n'est pas thread-safe, son partage direct entre requêtes HTTP concurrentes corromprait la session de persistance. Nous utilisons le pattern **ThreadLocal** dans **JPAUtil** pour lier chaque instance d'EntityManager au thread courant exécutant la requête HTTP. Chaque appelant dispose ainsi d'un gestionnaire de persistance isolé et sécurisé, détruit en fin de cycle de requête.

6 Infrastructure, Persistance et Couche Modèle

6.1 Conteneurisation Database (PostgreSQL et Docker)

La base de données PostgreSQL est configurée via un fichier Docker Compose facilitant le déploiement sur le port physique local 5436.

```
1 version: '3.8'
2 services:
3   db:
4     image: postgres:15
5     container_name: postgres_jee_absences
6     environment:
7       POSTGRES_DB: gestion_absences
8       POSTGRES_USER: postgres
9       POSTGRES_PASSWORD: postgres
10    ports:
11      - "5436:5432"
12    volumes:
13      - pgdata:/var/lib/postgresql/data
14
15 volumes:
16   pgdata:
```

Listing 1 – Fichier docker-compose.yml

6.2 Configuration de JPA (persistence.xml)

L'unité de persistance JPA configure le driver PostgreSQL JDBC et la génération automatique du schéma de tables.

```
1 <persistence xmlns="http://xmlns.jcp.org/xml/ns/persistence"
2   version="2.1">
3   <persistence-unit name="AbsencePU" transaction-type="
4     RESOURCE_LOCAL">
5     <provider>org.hibernate.jpa.HibernatePersistenceProvider<
6     /provider>
7     <class>com.gestionabsences.model.Etudiant</class>
8     <class>com.gestionabsences.model.Cours</class>
9     <class>com.gestionabsences.model.Seance</class>
10    <class>com.gestionabsences.model.Absence</class>
11    <class>com.gestionabsences.model.Professeur</class>
12    <properties>
13      <property name="javax.persistence.jdbc.driver" value=
14        "org.postgresql.Driver" />
15      <property name="javax.persistence.jdbc.url" value="
16        jdbc:postgresql://localhost:5436/gestion_absences" />
17      <property name="javax.persistence.jdbc.user" value="
18        postgres" />
19      <property name="javax.persistence.jdbc.password"
20        value="postgres" />
21    </properties>
22  </persistence-unit>
23 </persistence>
```

```

15         <property name="hibernate.dialect" value="org.
hibernate.dialect.PostgreSQL95Dialect" />
16         <property name="hibernate.hbm2ddl.auto" value="update
" />
17         <property name="hibernate.show_sql" value="true" />
18         <property name="hibernate.format_sql" value="true" />
19     </properties>
20 </persistence-unit>
21 </persistence>

```

Listing 2 – Fichier persistence.xml

6.3 Implémentation des Classes du Domaine (Modèles JPA)

Dans le cadre de l'architecture JPA, chaque table de la base de données PostgreSQL est représentée par une classe Java annotée (Entity). Afin d'assurer une séparation propre des préoccupations (Clean Architecture), ces entités se concentrent sur le mapping relationnel.

Voici la description conceptuelle de nos cinq entités principales :

1. **Etudiant** : Modélise un étudiant identifié de manière unique par son CNE. Il possède des attributs de connexion (email et mot de passe) et son nom complet.
2. **Professeur** : Représente l'enseignant responsable des appels. Il possède un nom, prénom, email et mot de passe.
3. **Cours** : Représente une matière enseignée, associée à un professeur unique via une relation de type plusieurs-à-un (ManyToOne).
4. **Seance** : Représente un créneau horaire de cours (date, heure de début, heure de fin) rattaché à un cours.
5. **Absence** : C'est l'entité centrale du système. Elle lie un étudiant à une séance de cours spécifique et porte les statuts de justification et de réclamation.

Pour illustrer le paramétrage JPA de notre projet, nous présentons ci-dessous un extrait de la classe `Absence.java` montrant la configuration des clés étrangères, de la contrainte d'unicité (un étudiant ne peut avoir qu'une seule absence par séance) et des nouveaux champs de modération :

```

1 @Entity
2 @Table(name = "absences", uniqueConstraints = {
3     @UniqueConstraint(columnNames = {"etudiant_id", "seance_id"})
4 })
5 public class Absence implements Serializable {
6     @Id
7     @GeneratedValue(strategy = GenerationType.IDENTITY)
8     private Long id;
9
10    @ManyToOne(fetch = FetchType.EAGER)
11    @JoinColumn(name = "etudiant_id", nullable = false)
12    private Etudiant etudiant;
13
14    @ManyToOne(fetch = FetchType.EAGER)
15    @JoinColumn(name = "seance_id", nullable = false)
16    private Seance seance;

```

```
17     @Column(name = "status_justification", length = 20)
18     private String statusJustification = "NONE"; // NONE, PENDING
19     , ACCEPTED, REJECTED
20
21     @Column(name = "status_reclamation", length = 20)
22     private String statusReclamation = "NONE"; // NONE, PENDING,
23     REJECTED
24 }
```

Listing 3 – Extrait JPA de la classe Absence.java

7 Gestion du Cycle de Vie JPA et EntityManager

7.1 Le Listener de Démarrage (JPAInitializer.java)

Tomcat étant un conteneur de servlets léger, il n'initialise pas nativement la persistance JPA au chargement. Nous y remédions en implémentant un `ServletContextListener` annoté `@WebListener`. Ce listener instancie l'unique `EntityManagerFactory` au démarrage du serveur et le ferme proprement à l'arrêt du contexte, évitant ainsi les fuites de ressources.

7.2 Thread Isolation des Managers (JPAUtil.java)

L'`EntityManager` n'étant pas thread-safe, nous utilisons la classe `ThreadLocal` de Java pour associer une instance d'`EntityManager` distincte à chaque thread de requête HTTP traitée par le pool de Tomcat.

La classe utilitaire `JPAUtil` centralise cette logique, permettant à toutes les classes DAO d'obtenir de façon sécurisée le gestionnaire de persistance lié au thread courant :

```
1 public class JPAUtil {
2     private static EntityManagerFactory emf;
3     private static final ThreadLocal<EntityManager>
4     threadLocalEntityManager = new ThreadLocal<>();
5
6     public static EntityManager getEntityManager() {
7         EntityManager em = threadLocalEntityManager.get();
8         if (em == null || !em.isOpen()) {
9             em = emf.createEntityManager();
10            threadLocalEntityManager.set(em);
11        }
12        return em;
13    }
14 }
```

Listing 4 – Gestion du ThreadLocal pour l'EntityManager dans JPAUtil.java

8 Couche de Persistance : Implémentation des DAOs

8.1 Gestion Manuelle des Transactions

En l'absence de conteneur d'injection (comme Spring ou EJB), nous gérons manuellement le cycle de vie de chaque transaction de persistance via l'interface `EntityTransaction` du standard JPA. Les étapes respectent scrupuleusement le patron suivant :

1. Récupération de l'`EntityManager` associé au thread courant par la méthode utilitaire `JPAUtil.getEntityManager()`.
2. Démarrage de la transaction via `tx.begin()`.
3. Exécution des opérations d'écriture avec `persist` ou `merge`.
4. Validation de l'état persistant avec `tx.commit()`.
5. En cas d'exception, annulation via `tx.rollback()` et remontée de l'erreur.
6. Libération finale de l'`EntityManager` dans le bloc `finally`.

8.2 Le Patron Data Access Object (DAO)

Afin de découpler la couche d'accès aux données du reste de l'application, nous avons développé trois classes DAO principales : `AbsenceDAO`, `EtudiantDAO` et `ProfesseurDAO`. Elles encapsulent les requêtes JPQL (Java Persistence Query Language) et les opérations de recherche par clé primaire.

Pour illustrer l'implémentation de nos requêtes complexes, voici un extrait de la classe `AbsenceDAO.java` montrant la récupération des requêtes en attente de modération (justifications ou réclamations) :

```
1 public class AbsenceDAO {
2     public List<Absence> findPendingRequests() {
3         EntityManager em = JPAUtil.getEntityManager();
4         try {
5             return em.createQuery(
6                 "SELECT a FROM Absence a WHERE a.
7                 statusJustification = 'PENDING' OR a.statusReclamation = '
8                 PENDING'",
9                 Absence.class
10            ).getResultList();
11        } finally {
12            JPAUtil.closeEntityManager();
13        }
14    }
15 }
```

Listing 5 – Extrait des requêtes JPQL dans `AbsenceDAO.java`

9 Couche Contrôleur : La Servlet Front-Controller

La Servlet centralisée `AbsenceServlet.java` gère à elle seule le cycle de routage de la plateforme, vérifiant l'authentification et dispatchant les actions.

9.1 Vérification des Rôles et Session Gate

Afin d'éviter tout accès illégitime, la Servlet implémente un filtre directement dans les méthodes `doGet` et `doPost` :

- Extraction de la session sans création (`getSession(false)`).
- Si null, redirection vers le login.
- Si existante, comparaison de l'attribut `userRole`.

9.2 Structure Majeure du Servlet de Contrôle

Le servlet de contrôle centralise la logique de routage. Il analyse le paramètre `action` passé en paramètre d'URL (pattern Front-Controller) et aiguille la requête vers la méthode de traitement dédiée.

Afin d'illustrer la gestion des rôles à la connexion et le téléversement de fichiers multipart sans framework externe, voici les extraits clés de traitement :

```
1 private void processLogin(HttpServletRequest request,
2   HttpServletResponse response)
3   throws ServletException, IOException {
4   String email = request.getParameter("email");
5   String password = request.getParameter("password");
6
7   // Authentification Admin
8   if ("admin@univ.fr".equals(email) && "admin".equals(password)
9   ) {
10    HttpSession session = request.getSession(true);
11    session.setAttribute("userRole", "ADMIN");
12    response.sendRedirect(request.getContextPath() + "/"
13    absences?action=list");
14    return;
15  }
16  // Traitement des roles PROFESSEUR et ETUDIANT via les DAOs
17  correspondants...
```

Listing 6 – Extrait de l'authentification unifiée dans `AbsenceServlet.java`

```
1 private void processStudentJustification(HttpServletRequest request,
2   HttpServletResponse response)
3   throws ServletException, IOException {
4   Long id = Long.parseLong(request.getParameter("id"));
5   Part filePart = request.getPart("pieceJustificative");
6   Absence abs = absenceDAO.findById(id);
7
8   if (abs != null && filePart != null && filePart.getSize() >
9   0) {
10    String fileName = filePart.getSubmittedFileName();
11    String uniqueName = System.currentTimeMillis() + "_" +
12    fileName;
13    String uploadPath = request.getServletContext().
14    getRealPath("/") + File.separator + "uploads";
```

```
12         filePart.write(uploadPath + File.separator + uniqueName);
13         abs.setDemandeJustifFilePath("uploads/" + uniqueName);
14         abs.setStatusJustification("PENDING");
15         absenceDAO.save(abs);
16     }
17     response.sendRedirect(request.getContextPath() + "/absences?
18     action=studentDashboard");
19 }
```

Listing 7 – Gestion de l'upload de justificatif dans AbsenceServlet.java

10 Gestion d'État : Sessions & Cookies

10.1 Statelessness HTTP et HttpSession

Le protocole HTTP est par nature sans état (stateless); chaque requête du client est traitée de façon totalement indépendante de la précédente. Pour maintenir la continuité de l'accès au tableau de bord, la plateforme s'appuie sur la classe standard `HttpSession`.

Lorsqu'un utilisateur s'authentifie, un identifiant de session unique (JSESSIONID) est stocké en mémoire serveur et partagé avec le navigateur sous forme de cookie temporaire. Ce cookie est transmis à chaque requête, permettant au serveur de charger les attributs associés à la session.

10.2 Persistance et Remplissage via Cookies

Pour améliorer le confort d'utilisation, un Cookie persistant est créé sur le client si la case "Se souvenir de moi" est cochée.

```
1 private void showLoginForm(HttpServletRequest request,
2     HttpServletResponse response)
3     throws ServletException, IOException {
4     Cookie[] cookies = request.getCookies();
5     if (cookies != null) {
6         for (Cookie c : cookies) {
7             if ("rememberedAdmin".equals(c.getName())) {
8                 request.setAttribute("rememberedAdmin", c.
9                 getValue());
10                break;
11            }
12        }
13    }
14    request.getRequestDispatcher("/WEB-INF/views/login.jsp").
15    forward(request, response);
16 }
```

Listing 8 – Lecture des cookies au chargement du login - AbsenceServlet.java

11 Téléversement de Fichiers & Exportations

11.1 Téléversement avec Multipart Config

Pour gérer les pièces jointes des étudiants de manière sécurisée, la Servlet est configurée avec l'annotation `@MultipartConfig` qui définit des contraintes de taille sur les téléversements. Les justificatifs sont sauvés dans un dossier physique `uploads/` au sein du répertoire de déploiement web.

11.2 Exportation CSV avec Signature BOM

L'exportation en format CSV est codée en écrivant directement dans le flux de sortie HTTP (`response.getOutputStream()`). Pour éviter que Microsoft Excel n'altère l'affichage des caractères accentués des noms d'étudiants (é, à, ç), nous écrivons la signature BOM UTF-8 (0xEF, 0xBB, 0xBF) avant d'initier l'écriture des chaînes de caractères.

12 Interface Utilisateur (UI) & CSS Glassmorphism

12.1 Philosophie du Design System

Pour proposer une esthétique digne des meilleures applications Web, nous avons défini un design Glassmorphic (effet verre dépoli) basé sur des couleurs sombres, des reflets lumineux et des flous d'arrière-plan.

```
1 :root {
2   --bg-primary: #0b0f19;
3   --accent-purple: #8b5cf6;
4   --accent-blue: #3b82f6;
5   --success: #10b981;
6   --danger: #ef4444;
7   --text-primary: #f8fafc;
8   --glass-bg: rgba(30, 41, 59, 0.45);
9   --glass-border: rgba(255, 255, 255, 0.08);
10 }
11 body {
12   background-color: var(--bg-primary);
13   color: var(--text-primary);
14   font-family: 'Inter', sans-serif;
15 }
16 .card {
17   background: var(--glass-bg);
18   backdrop-filter: blur(16px);
19   border: 1px solid var(--glass-border);
20   border-radius: 12px;
21 }
```

Listing 9 – Design Tokens - style.css

12.2 Filtrage client temps réel (Vanilla JS)

Le filtrage en temps réel dans les tableaux d'absences est effectué en JavaScript côté navigateur pour une interactivité instantanée.

```

1 function filterAbsences() {
2     var query = document.getElementById("searchInput").value.
      toLowerCase();
3     var filter = document.getElementById("statusFilter").value;
4     var rows = document.querySelectorAll("table tbody tr");
5
6     rows.forEach(function(row) {
7         var nom = row.cells[1].textContent.toLowerCase();
8         var cours = row.cells[2].textContent.toLowerCase();
9         var status = row.cells[4].textContent.toLowerCase();
10
11         var matchesQuery = nom.indexOf(query) > -1 || cours.
      indexOf(query) > -1;
12         var matchesStatus = filter === "ALL" ||
13             (filter === "JUSTIF" && status.indexOf("justifi") >
      -1) ||
14             (filter === "NONJUSTIF" && status.indexOf("non
      justifi") > -1);
15
16         row.style.display = (matchesQuery && matchesStatus) ? ""
      : "none";
17     });
18 }

```

Listing 10 – Filtrage interactif

13 Structure Détaillée des Vues (JSPs)

La couche de présentation utilise des pages JSP (JavaServer Pages) associées à la bibliothèque de balises standard JSTL (JavaServer Pages Standard Tag Library). Cela permet d'éviter l'écriture de code Java brut (scriptlets) au sein des fichiers HTML, préservant ainsi le modèle MVC.

13.1 Espace Étudiant - student-dashboard.jsp

Cette JSP présente les statistiques de l'étudiant connecté et liste ses absences à travers un tableau interactif. L'étudiant peut initier deux processus : le dépôt de justificatif et la réclamation.

La directive JSTL ci-dessous illustre le rendu conditionnel du bouton d'action selon le statut de l'absence :

```

1 <c:forEach var="abs" items="${absences}">
2     <tr>
3         <td>${abs.formattedDateSaisie}</td>
4         <td>${abs.seance.cours.nomCours}</td>
5         <td>${abs.estJustifiee ? 'Justifiee' : 'Non Justifiee'}</
      td>

```

```

6         <td>
7             <c:if test="${not abs.estJustifiee}">
8                 <button onclick="openJustify(${abs.id})">
9                     Justifier</button>
10                </c:if>
11            </td>
12        </tr>
13    </c:forEach>

```

Listing 11 – Rendu conditionnel avec JSTL dans student-dashboard.jsp

13.2 Espace Enseignant - prof-dashboard.jsp

L'espace professeur implémente la feuille d'appel dynamique. L'enseignant choisit une séance, coche les étudiants absents et soumet le formulaire. La liste des étudiants est parcourue dynamiquement, et les identifiants cochés sont transmis en tableau de paramètres HTTP au servlet de contrôle :

```

1 <c:forEach var="st" items="${etudiants}">
2     <div class="student-item">
3         <span>${st.nomCompleet}</span>
4         <input type="checkbox" name="absentStudents" value="${st.
5             id}">
6     </div>
7 </c:forEach>

```

Listing 12 – Boucle de sélection des étudiants pour l'appel

13.3 Espace Administration - admin-requests.jsp

Le portail de modération liste toutes les demandes reçues avec leurs motifs et les liens de téléchargement des justificatifs stockés en serveur. L'administrateur peut accepter ou rejeter chaque demande en un clic :

```

1 <c:forEach var="abs" items="${pendingAbsences}">
2     <div class="request-card">
3         <h3>${abs.etudiant.nomCompleet} - ${abs.seance.cours.
4             nomCours}</h3>
5         <p>Motif: ${abs.motifDemandeJustif}</p>
6         <a href="${pageContext.request.contextPath}/absences?
7             action=resolveJustification&id=${abs.id}&decision=accept">
8             Accepter</a>
9         <a href="${pageContext.request.contextPath}/absences?
10            action=resolveJustification&id=${abs.id}&decision=refuse">
11            Refuser</a>
12     </div>
13 </c:forEach>

```

Listing 13 – Actions de modération admin

14 Déploiement et Configuration

14.1 Configuration de déploiement (web.xml)

Le fichier ‘web.xml’ configure l’encodage des requêtes et le mapping d’URL racine ‘/absences’ pour diriger le trafic vers notre Servlet de contrôle unique.

14.2 Maven pom.xml

Les dépendances Maven intègrent les APIs de Servlets, JSPs, JSTL, Hibernate Core et le connecteur JDBC de PostgreSQL.

15 Scénarios d’Exécution et Recette de Tests

Cette section présente la validation fonctionnelle de la plateforme à travers les différents cas d’utilisation réels et les captures d’écran associées.

15.1 Test 1 : Authentification multi-rôle

L’accès à l’application est sécurisé par un portail d’authentification unique aiguillant les utilisateurs selon leur rôle en base de données.

- **Action** : Saisie des identifiants (administrateur, professeur ou étudiant) dans le formulaire de connexion.
- **Résultat attendu** : Redirection vers le dashboard correspondant.

15.2 Test 2 : Mémorisation de session par Cookies

Pour améliorer l’expérience utilisateur, l’option « Se souvenir de moi » stocke un cookie sécurisé sur le client.

- **Action** : Activation de la case « Se souvenir de moi » lors du login et vérification de la persistance après fermeture du navigateur.
- **Résultat attendu** : Création du cookie correspondant et reconnexion automatique.

15.3 Test 3 : Saisie d’appel interactif par le Professeur

L’enseignant peut sélectionner sa séance active et réaliser l’appel rapidement.

- **Action** : Sélection de la séance dans la liste déroulante, cochage des étudiants absents, et soumission.
- **Résultat attendu** : Création en base de données des entrées d’absences correspondantes.

15.4 Test 4 : Tableau de bord Étudiant et suivi personnel

L’étudiant accède en temps réel à l’état de son assiduité et peut lancer le workflow de justification ou de réclamation.

- **Action** : Consultation de la page d’accueil étudiante.

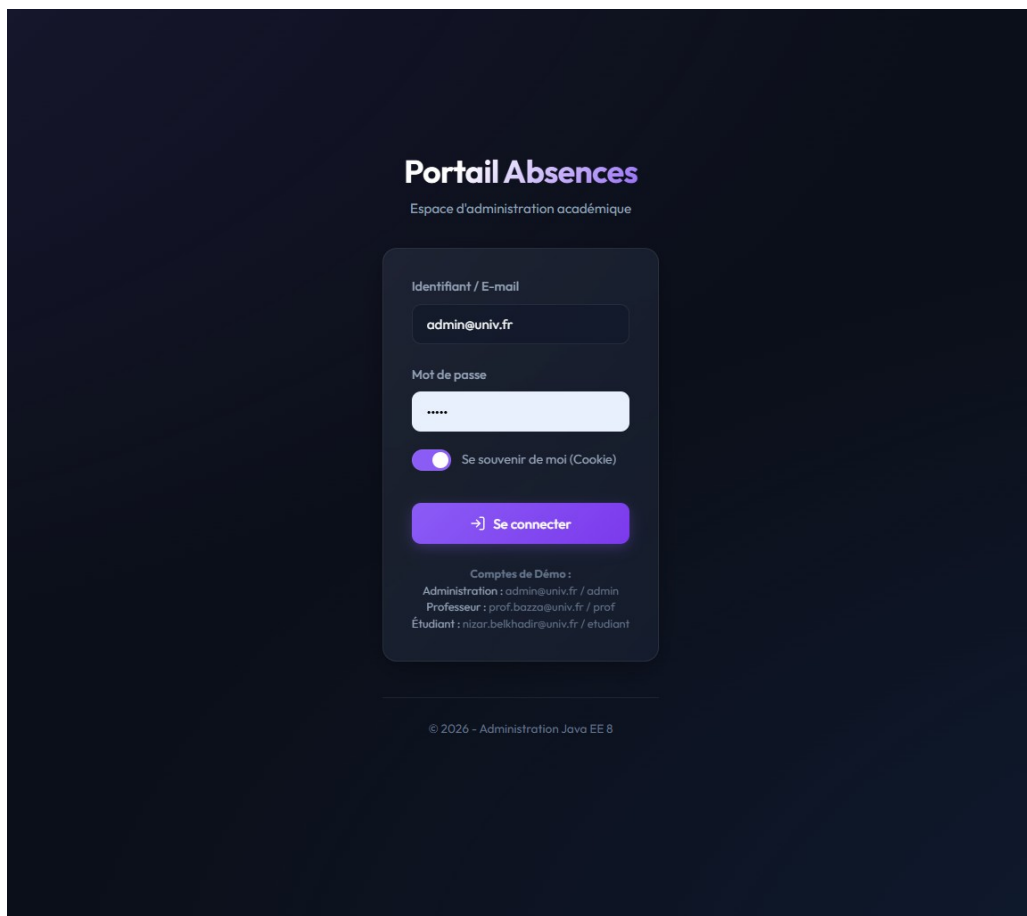


FIGURE 4 – Interface de Connexion Multi-Rôles

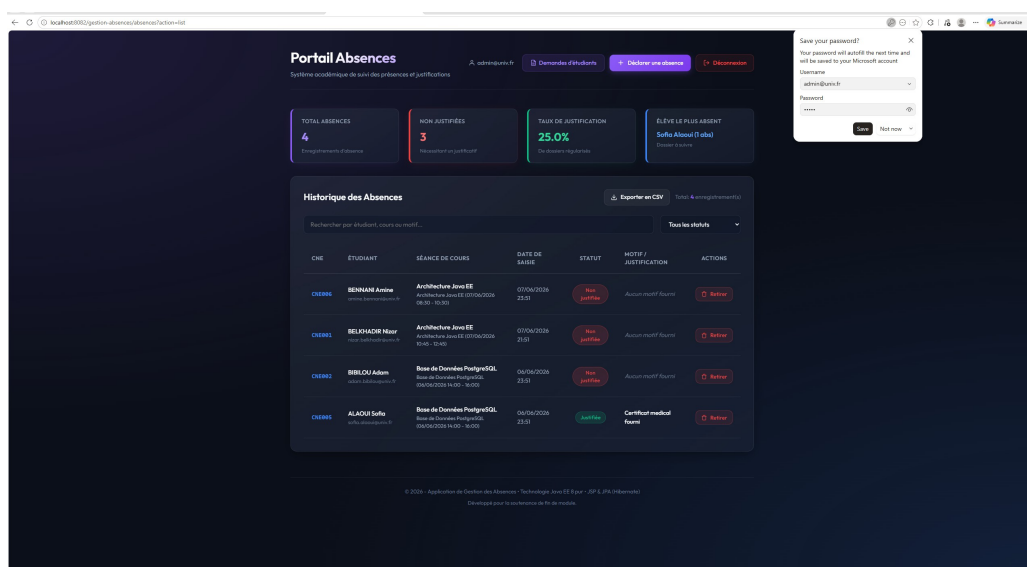


FIGURE 5 – Persistance d'authentification par Cookie HTTP

Espace Professeur

Bienvenue, Houssam Bazza

Enseignant [Déconnexion](#)

Saisie d'Appel (Faire l'appel)

Sélectionnez une séance et cochez uniquement les étudiants absents.

Séance d'Enseignement

-- Choisir une séance --

Liste des étudiants de la classe

Rechercher un étudiant par nom/CNE...

Alaoui Sofya CNE: CNE003 sofya.alaoui@univ.fr PRÉSENT ☐	Asaas Adam CNE: CNE003 adam.asaas@univ.fr PRÉSENT ☐	Belkhadir Nizar CNE: CNE001 nizar.belkhadir@univ.fr PRÉSENT ☐
Benjelloun Ali CNE: CNE034 ali.benjelloun@univ.fr PRÉSENT ☐	Benkrane Ghita CNE: CNE024 ghita.benkrane@univ.fr PRÉSENT ☐	Bennani Amine CNE: CNE006 amine.bennani@univ.fr PRÉSENT ☐
Berrada Nassima CNE: CNE033 nassima.berrada@univ.fr PRÉSENT ☐	Bibilou Adam CNE: CNE002 adam.bibilou@univ.fr PRÉSENT ☐	Boutaleb Imane CNE: CNE018 imane.boutaleb@univ.fr PRÉSENT ☐
Cherkaoui Hamza CNE: CNE029 hamza.cherkaoui@univ.fr PRÉSENT ☐	Chraïbi Salma CNE: CNE022 salma.chraïbi@univ.fr PRÉSENT ☐	Daoudi Mehdi CNE: CNE025 mehdi.daoudi@univ.fr PRÉSENT ☐
El Amri Youssef CNE: CNE022 youssef.elamri@univ.fr PRÉSENT ☐	El Fassi Yassine CNE: CNE007 yassine.elfassi@univ.fr PRÉSENT ☐	Fadli Reda CNE: CNE017 reda.fadli@univ.fr PRÉSENT ☐
Filali Anass CNE: CNE019 anass.filali@univ.fr PRÉSENT ☐	Gharbi Sara CNE: CNE030 sara.gharbi@univ.fr PRÉSENT ☐	Haddad Adnane CNE: CNE013 adnane.haddad@univ.fr PRÉSENT ☐
Jebbar Othmane CNE: CNE011 othmane.jebbar@univ.fr PRÉSENT ☐	Kabbaj Omar CNE: CNE015 omar.kabbaj@univ.fr PRÉSENT ☐	Kadiri Tarik CNE: CNE021 tarik.kadiri@univ.fr PRÉSENT ☐
Lahlou Adil CNE: CNE027 adil.lahlou@univ.fr PRÉSENT ☐	Mansouri Hind CNE: CNE028 hind.mansouri@univ.fr PRÉSENT ☐	Mernisi Fouad CNE: CNE026 fouad.mernisi@univ.fr PRÉSENT ☐

FIGURE 6 – Feuille d'Appel Interactive de l'Enseignant

- **Résultat attendu** : Affichage précis du nombre total d'absences et des détails de chaque séance.

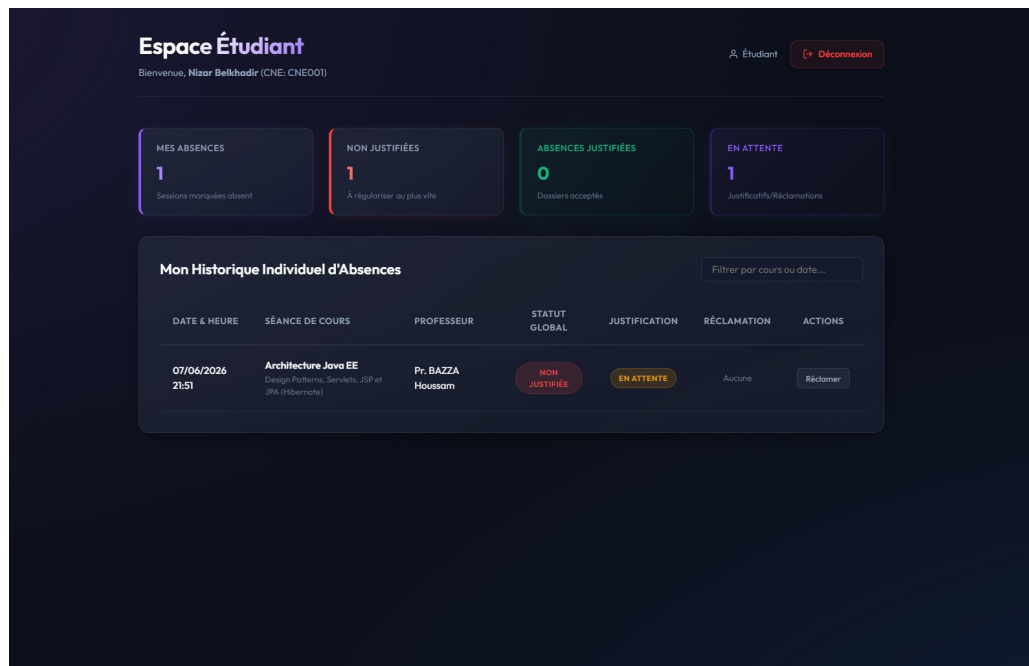


FIGURE 7 – Tableau de Bord Étudiant et Statut des Absences

15.5 Test 5 : Modération administrative des justificatifs et réclamations

L'administrateur arbitre et statue sur les demandes de régularisation déposées.

- **Action** : Examen des justificatifs médicaux téléversés et clic sur « Accepter » ou « Refuser ».
- **Résultat attendu** : Mise à jour automatique du statut de l'absence en base de données.

15.6 Tests Unitaires Automatisés (JUnit)

Pour garantir la robustesse de l'application et éviter toute régression lors des phases de refactoring, nous avons intégré des tests unitaires automatisés basés sur le framework **JUnit 4**. Ces tests permettent de valider le comportement logique de nos modèles et utilitaires en isolation complète.

Trois classes de test ont été implémentées dans le répertoire `src/test/java` :

- **EtudiantTest** : Vérifie le bon fonctionnement du constructeur, des accesseurs (Getters/Setters) et la méthode utilitaire `getNomCompleet()` qui convertit automatiquement le nom de famille en majuscules.
- **SeanceTest** : Valide le formatage personnalisé de la date (dd/MM/yyyy) et la génération dynamique du libellé de séance combinant le nom du cours, la date et les horaires.
- **JPAUtilTest** : Teste la sécurité d'accès de la classe utilitaire de persistance en vérifiant que toute demande d'accès à l'EntityManager lève une exception `IllegalStateException` si la fabrique globale n'a pas encore été initialisée par le listener de démarrage.

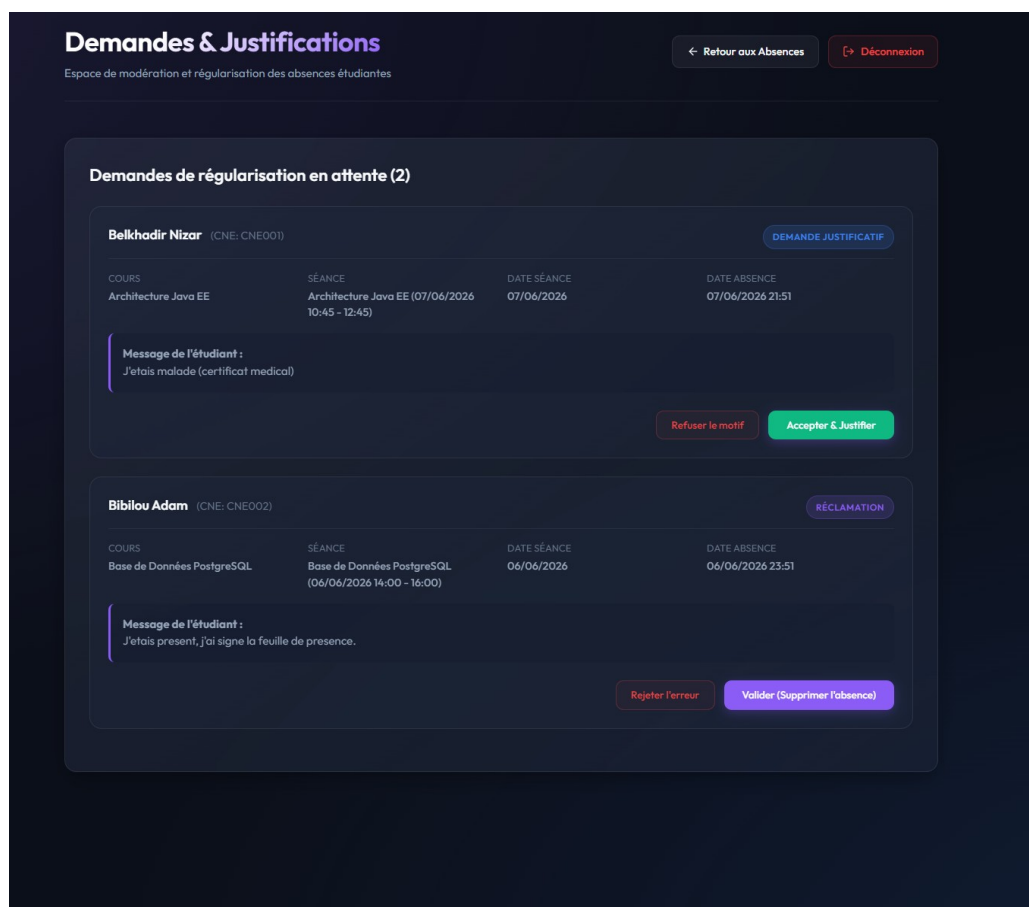


FIGURE 8 – Portail d'Arbitrage et de Modération Admin

```
1 package com.gestionabsences;
2
3 import com.gestionabsences.model.Etudiant;
4 import org.junit.Test;
5 import static org.junit.Assert.*;
6
7 public class EtudiantTest {
8     @Test
9     public void testGetNomCompleet() {
10         Etudiant etudiant = new Etudiant("CNE999", "belkhadir", "
nizar", "nizar@univ.fr");
11         assertEquals("BELKHADIR nizar", etudiant.getNomCompleet())
12     ;
13 }
```

Listing 14 – Extrait des tests unitaires de validation du modèle Etudiant

16 Conclusion et Perspectives

L'architecture robuste de cette plateforme multi-rôles en Java EE 8 pur démontre la pertinence des standards natifs pour concevoir des applications modulaires et sécurisées. Par l'apprentissage des transactions JPA, de la sécurité par session et de l'interactivité native client, nous avons consolidé nos compétences fondamentales en ingénierie logicielle d'entreprise.